

# GANDEHOU

## Documentation Backend

Securisation des transactions foncieres au Benin par ancrage cryptographique sur Bitcoin.  
Documentation complete des livrables backend, mise en regard avec le dossier de cadrage original.

---

**Reference projet :** `uhfyofjxolhpunpbdefq.supabase.co`  
**Date :** Cotonou . 30 juin 2026  
**Source de cadrage :** Gandehou\_Dossier\_Foncier\_Bitcoin\_Benin\_2026.pdf

# 1. Resume executif

---

Gandehou est une couche de preuve d'antériorité et d'intégrité pour les transactions foncières au Bénin. Chaque document (attestation provisoire, convention finale, pièce annexe) est haché en SHA-256 puis horodaté sur Bitcoin via le protocole OpenTimestamps. La vérification est publique, gratuite, sans création de compte, et survivante (n'importe quel outil OpenTimestamps tiers peut décoder une preuve, même si nos serveurs disparaissent).

Le backend de Gandehou est aujourd'hui opérationnel sur l'infrastructure Supabase (Postgres, Auth, Storage, Edge Functions Deno). L'ancrage Bitcoin a été testé end-to-end avec une preuve réellement soumise aux calendriers OpenTimestamps publics. La détection de falsification fonctionne et a été validée par un scénario de PDF altéré après ancrage.

## Ce qui est livré

- Schema SQL Postgres avec 5 tables, RLS active, 2 triggers d'audit automatique
- 9 services TypeScript typés couvrant tout le cycle de vie d'un dossier foncier
- Moteur de règles ANDF aligné sur le Code Foncier et Domaniai (CFD)
- Wrapper OpenTimestamps isomorphe (Node et Deno) sur la bibliothèque @otskit/client
- 3 Edge Functions déployées en production : anchor-document, upgrade-ots, verify-proof
- Cron pg\_cron de mise à jour automatique des preuves toutes les 30 minutes
- Test end-to-end automatisé en 12 étapes, dont scénario de falsification vérifié
- Documentation d'intégration BACKEND.md pour le développeur frontend

## Ce qui n'est PAS livré (et c'est volontaire)

Conformément au dossier de cadrage (sections 6 et 7.2 : 'Ce que Gandehou ne doit pas prétendre être'), nous ne prétendons pas remplacer le Notaire ni l'ANDF. Notre couche se positionne en complément de confiance du circuit légal existant, pas en substitut.

- Auth par OTP SMS réelle : simulée en localStorage (provider tiers type Twilio non intégré, conformément au choix hackathon)
- Intégration directe au numéro unique parcellaire et au CEC : documentée comme roadmap post-hackathon (section 7.3 du cadrage)
- RLS strictes par rôle : volontairement permissives (using(true)) pour la démonstration; à durcir en V1 production
- Génération PDF côté serveur : conserve côté client via pdf-lib (suffisant pour le hackathon)

## 2. Graphique d'avancement

Avancement par composant, mesure par rapport au perimetre defini dans le dossier de cadrage original.

### Couche persistance (Supabase Postgres)

|                                      |             |      |       |
|--------------------------------------|-------------|------|-------|
| Schema SQL (5 tables, triggers, RLS) | <div></div> | 100% | comp  |
| Index et contraintes metier          | <div></div> | 100% | comp  |
| Triggers d'audit automatique         | <div></div> | 100% | comp  |
| RLS strictes par role                | <div></div> | 25%  | permi |

### Couche services applicatifs (TypeScript)

|                                    |             |      |       |
|------------------------------------|-------------|------|-------|
| CRUD profils, dossiers, documents  | <div></div> | 100% | comp  |
| Storage (3 buckets, helpers spec.) | <div></div> | 100% | comp  |
| Audit trail lecture                | <div></div> | 100% | comp  |
| Moteur regles ANDF                 | <div></div> | 90%  | 4 reg |
| Chainage de hash auto              | <div></div> | 100% | comp  |
| Action rejet dossier               | <div></div> | 100% | comp  |

### Coeur Bitcoin demonstrable

|                                  |             |      |        |
|----------------------------------|-------------|------|--------|
| Module OpenTimestamps isomorphe  | <div></div> | 100% | comp   |
| Edge Function anchor-document    | <div></div> | 100% | deplo  |
| Edge Function upgrade-ots + cron | <div></div> | 100% | deplo  |
| Edge Function verify-proof       | <div></div> | 100% | deplo  |
| Detection de falsification       | <div></div> | 100% | valide |

### Outillage et documentation

|                                 |             |      |      |
|---------------------------------|-------------|------|------|
| Test e2e automatise (12 etapes) | <div></div> | 100% | comp |
| Documentation BACKEND.md        | <div></div> | 100% | comp |
| Documentation deploiement       | <div></div> | 100% | comp |

#### BILAN GLOBAL

Backend hackathon : 100% livre. Backend production V1 : ~70% (manque RLS strictes, validation serveur des regles ANDF, upload pieces justificatives, notifications email).

## 3. Conformite au dossier de cadrage

Exigence par exigence du dossier de cadrage original, mise en regard de ce qui a ete effectivement livre.

### 3.1 Fonctionnalites coeur (section 7.1 du dossier)

| Exigence dossier                          | Statut | Livvable                                |
|-------------------------------------------|--------|-----------------------------------------|
| Saisie citoyenne (form. multi-etapes)     | OK     | Schema dossiers + service createDossier |
| Generation PDF avec filigrane + QR        | OK     | lib/pdf.ts cote client (existant)       |
| Calcul SHA-256 a chaque etape             | OK     | packages/ledger/hash.ts                 |
| Ancrage Bitcoin via OpenTimestamps        | OK     | Edge Function anchor-document           |
| Chainage des hashes entre etapes          | OK     | createChainedDocument avec hash_parent  |
| Page de verification publique             | OK     | Service verify.ts + verifyFileDeep      |
| Verdict visuel 3 etats (vert/jaune/rouge) | OK     | ots_status: confirmed/pending/mismatch  |
| Tableau de bord par role                  | OK     | Service listDossiersAvecDernierDocument |

### 3.2 Architecture technique (section 8 du dossier)

| Exigence dossier                        | Statut    | Livvable                               |
|-----------------------------------------|-----------|----------------------------------------|
| Postgres pour dossiers, statuts, users  | OK        | supabase/schema.sql                    |
| Supabase Auth (email + OTP)             | OK        | Email reel; phone simule (hackathon)   |
| Row Level Security par role             | Partiel   | RLS active, politiques permissives     |
| Supabase Storage (provisoire/definitif) | OK        | 3 buckets + helpers storage.ts         |
| Edge Functions Deno                     | OK        | 3 fonctions deployees                  |
| pg_cron upgrade preuves OTS             | OK        | Job gandehou-upgrade-ots toutes 30 min |
| javascript-opentimestamps               | Substitue | @otskit/client (lib moderne sans deps) |

### 3.3 Modele de donnees (section 10 du dossier)

Le schema livre suit exactement la section 10 du dossier, avec 2 ajouts pragmatiques justifies.

| Table du dossier       | Statut | Notes                                                    |
|------------------------|--------|----------------------------------------------------------|
| profiles               | OK     | FK auth.users souple (chef demo)                         |
| dossiers               | OK +   | Ajout flag_etrangeur_zone_rurale + flag_superficie_seuil |
| documents              | OK     | Ajout ots_block_height + ots_confirmed_at                |
| dossier_status_history | OK     | Auto-remplie par trigger                                 |
| otp_sessions           | OK     | Conservee pour coherence (placeholder hackathon)         |

### 3.4 Statuts a afficher honnetement (section 9.2 du dossier)

Le dossier impose explicitement de ne JAMAIS pretendre qu'un statut est confirmed s'il est pending.

| Statut interne | Statut UI attendu             | Implementation                         |
|----------------|-------------------------------|----------------------------------------|
| pending        | En attente Bitcoin (jaune)    | Default a l'insert + apres ancrage OTS |
| confirmed      | Ancre sur Bitcoin (vert)      | Cron upgrade-ots OU verify-proof       |
| mismatch       | Modifie apres ancrage (rouge) | verify-proof si hash divergent         |

## 4. Detail des livrables par couche

### 4.1 Schema SQL (supabase/schema.sql)

5 tables, 2 triggers, RLS activee partout. Source unique de verite, miroir des types TypeScript.

```
create table profiles (id, role, full_name, email, phone, commune, ...)
create table dossiers (id, statut, vendeur_*, acheteur_*, zone, ...)
create table documents (id, dossier_id, sha256, hash_parent, ots_status, ...)
create table dossier_status_history (id, ancien_statut, nouveau_statut, ...)
create table otp_sessions (id, telephone, code_hash, expires_at, ...)
```

```
trigger touch_updated_at          -> tient dossiers.updated_at a jour
trigger log_status_change         -> historise tous les changements de statut
```

### 4.2 Services TypeScript (apps/web/src/services/)

| Fichier        | Role                                                     |
|----------------|----------------------------------------------------------|
| profiles.ts    | CRUD profils + getCurrentOfficialProfile                 |
| dossiers.ts    | CRUD + filtres par role + changerStatut + rejeterDossier |
| documents.ts   | CRUD + recherche par sha256 + createChainedDocument      |
| storage.ts     | Upload PDF + .ots dans buckets, helpers specialises      |
| history.ts     | Lecture audit trail + annotation manuelle                |
| regles-andf.ts | Moteur juridique pur (4 regles CFD)                      |
| anchor.ts      | Invocation Edge Function anchor-document + upgrade       |
| verify.ts      | Verdict 3 etats + verification crypto deep               |
| index.ts       | Barrel d'imports unique                                  |

### 4.3 Module OpenTimestamps (packages/ledger/)

Wrapper isomorphe sur @otskit/client, library moderne zero-dependance. Exporte 3 fonctions pures :

- stampHash(sha256Hex) : Promise<StampResult> -- cree une preuve pending
- upgradeProof(proofBytes) : Promise<UpgradeResult> -- tente le passage pending vers confirmed
- verifyProof(proofBytes, hash) : Promise<VerifyResult> -- verdict typee fail-closed

#### Pourquoi @otskit/client

La bibliotheque javascript-opentimestamps citee dans le dossier original n'est plus maintenue depuis 2019 (deps web3 0.18, keccak 1.4 incompilables sur Windows recent). Substitution par @otskit/client (2026, zero-dep, TypeScript strict). L'API publique de notre wrapper reste identique : aucun impact sur le code applicatif.

## 5. Edge Functions Supabase

3 fonctions Deno deployees sur le projet uhfyofjxolhpunpbdefq. Endpoint base : <https://uhfyofjxolhpunpbdefq.functions.supabase.co/>

### 5.1 anchor-document

Ancrer un PDF deja uploadé sur Bitcoin via OpenTimestamps. Etapes internes :

- Lit la ligne documents (storage\_bucket + storage\_path + sha256)
- Idempotence : si ots\_proof\_path deja renseigne, renvoie l'etat sans re-ancrer
- Telecharge le PDF depuis Storage
- Recalcule SHA-256 cote serveur (le client est hostile par defaut)
- Verifie la coherence entre le hash en base et le hash recalcule
- Appelle OpenTimestamps stamp() : contacte les calendriers publics
- Uploade la preuve .ots dans le bucket ots-proofs
- Met a jour documents.ots\_proof\_path + ots\_status = 'pending'

```
POST /anchor-document
Body: { documentId: string }
Response 200: { ok: true, status: "pending", hash, proofPath, message }
Response 404: { ok: false, error: "Document XXX introuvable" }
Response 409: { ok: false, error: "Hash divergent...", dbHash, fileHash }
```

### 5.2 upgrade-ots

Tache batch planifiee toutes les 30 minutes par pg\_cron. Pour chaque document pending :

- Telecharge la preuve .ots actuelle
- Appelle OpenTimestamps upgrade() : tentative de confirmation Bitcoin
- Si confirmed : reuploadé la preuve enrichie + met a jour ots\_status + ots\_block\_height
- Si still pending : pas d'action, sera retente au prochain cycle
- Isolation par document : un echec n'interrompt pas les autres

```
POST /upgrade-ots
Body: {} (declenche par pg_cron, mais appellable manuellement)
Response: { ok: true, stats: { scanned, upgraded, stillPending, errors[] } }
```

### 5.3 verify-proof

Verification cryptographique reelle. Difference critique avec services/verify.ts :

- services/verify.ts (cote client) : compare juste un hash en base (instantane)
- verify-proof (cote serveur) : telecharge .ots, valide cryptographiquement contre Bitcoin
- C'est ce niveau de verification que le jury Bitcoin attend
- Effet de bord important : si mismatch detecte, met a jour ots\_status='mismatch' en base

```
POST /verify-proof
Body: { documentId } ou { sha256 }
Response 200: { ok: true, verdict: "confirmed"|"pending"|"mismatch"|"invalid", ... }
Confirmed: { blockHeight, bitcoinTimestamp }
Mismatch: { expectedHash, actualHash, reason }
```



## 6. Workflow metier complet

---

Cycle de vie d'une transaction fonciere dans Gandehou, du depot par le chef de quartier jusqu'a la validation finale par la Mairie.

### Etape 1 - Saisie citoyenne (statut: brouillon)

- Le chef de quartier remplit un stepper avec les donnees du dossier
- Le moteur ANDF evalue les regles en temps reel (alertes UI)
- Si bloquant (ex: etranger en zone rurale) : soumission refusee
- createDossier() insere la ligne + calcule les flags ANDF automatiquement
- Le trigger SQL log\_status\_change historise la creation

### Etape 2 - Generation de l'attestation provisoire (statut: atteste\_cq)

- PDF genere cote client via lib/pdf.ts (filigrane, QR code, donnees du dossier)
- Hash SHA-256 calcule via Web Crypto API
- createDocument() insere la ligne documents (type=attestation\_provisoire)
- uploadPdfProvisoire() pousse le PDF dans Storage
- anchorDocument() appelle l'Edge Function anchor-document
- ots\_status passe a 'pending' + .ots cree dans ots-proofs
- changerStatut(id, 'atteste\_cq')

### Etape 3 - Validation Mairie (statut: valide\_mairie)

- L'agent Mairie consulte le dashboard filtre sur sa commune
- Il peut soit valider, soit rejeter (rejeterDossier avec motif)
- Si validation : nouveau PDF (convention finale) genere
- createChainedDocument() : hash\_parent rempli avec le sha256 de l'attestation provisoire
- Refus si parent non encore ancre (garde-fou crypto)
- Upload + ancrage de la convention finale
- changerStatut(id, 'valide\_mairie')

### Etape 4 - Verification publique (sans authentication)

- Mode A : drag-and-drop du PDF -> sha256OfFile() -> verifyBySha256()
- Mode B : scan QR code -> verifyByDocumentId()
- Verdict instantane (lookup en base)
- En arriere-plan : verifyFileDeep() appelle verify-proof pour validation crypto reelle
- Si verify-proof detecte mismatch : ots\_status='mismatch' propage en base pour les futurs lookups

### Etape 5 - Cycle de fond (cron toutes les 30 min)

- pg\_cron appelle upgrade-ots automatiquement
- Toutes les preuves en pending sont scannees
- Celles qui ont une attestation Bitcoin disponible passent a confirmed
- Mise a jour de ots\_block\_height + ots\_confirmed\_at

## 7. Test end-to-end automatisé

Le fichier `supabase/test-e2e.mjs` valide la chaîne complète sur l'infrastructure réelle Supabase + OpenTimestamps. 12 étapes, idempotent, produit une sortie colorée.

### Procédure

```
# 1. Variables d'env dans supabase/.env.local
SUPABASE_URL=https://uhfyofjxolhpunpbdefq.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...

# 2. Lancement
node supabase/test-e2e.mjs
```

### Les 12 étapes

| #  | Étape                                 | Validation                     |
|----|---------------------------------------|--------------------------------|
| 1  | Création dossier test en base         | INSERT + UUID retourne         |
| 2  | Génération PDF en mémoire             | Bytes valides                  |
| 3  | Calcul SHA-256                        | 64 chars hex                   |
| 4  | Upload bucket documents-provisaires   | Pas d'erreur Storage           |
| 5  | Insert ligne documents                | Default ots_status=pending     |
| 6  | Appel anchor-document                 | proofPath retourne, 5-30s reel |
| 7  | Vérification ots_status en base       | pending + ots_proof_path       |
| 8  | Présence du .ots dans ots-proofs      | Download OK                    |
| 9  | Appel upgrade-ots                     | stillPending=N (normal)        |
| 10 | verify-proof (vérification crypto)    | verdict=pending attendu        |
| 11 | Simulation falsification (PDF altéré) | verdict=mismatch DETECTÉ       |
| 12 | Rejet dossier + audit trail           | litige + 2 entrées history     |

#### SCENARIO KILLER DU PITCH

L'étape 11 est ton scénario de démonstration clé : tu uploades un PDF différent à la place de l'original, et le système détecte que le hash recalculé ne correspond plus à la preuve ancrée. Le verdict passe à 'mismatch' et ots\_status est propagé en base. C'est exactement le récit 'Maman Chantal détecte la double vente' du README.

## 8. Infrastructure de deployment

### Etat actuel

| Composant                     | Etat     | Reference             |
|-------------------------------|----------|-----------------------|
| Projet Supabase               | Actif    | uhfyofjxolhpunpbdefq  |
| Schema SQL execute            | OK       | 5 tables + 2 triggers |
| Extension pg_cron             | Activee  | Database > Extensions |
| Extension pg_net              | Activee  | Database > Extensions |
| Edge Function anchor-document | Deployee | Live + testee         |
| Edge Function upgrade-ots     | Deployee | Live + testee         |
| Edge Function verify-proof    | Deployee | Live + testee         |
| Bucket documents-provisoires  | Cree     | Public read           |
| Bucket documents-definitifs   | Cree     | Public read           |
| Bucket ots-proofs             | Cree     | Public read           |
| Cron gandehou-upgrade-ots     | Active   | * / 30 * * * *        |

### Commandes de redeploiement

```
# Si modification d'une Edge Function
supabase functions deploy anchor-document
supabase functions deploy upgrade-ots
supabase functions deploy verify-proof

# Si modification du schema SQL
# (manuel via Dashboard > SQL Editor)
```

## 9. Limites assumees et roadmap V1 production

Conformement a la philosophie du dossier de cadrage (section 6 : 'Ce que Gandehou doit pretendre etre'), nous documentons honnetement les limites du livrable hackathon et le chemin vers la V1 production.

### 9.1 Limites volontaires du hackathon

- Auth citoyens en localStorage : aucun provider SMS reel (mentionne explicitement dans le dossier section 7.2)
- RLS permissives (using(true)) : les policies sont activees mais pas restrictives par role
- Validation des regles ANDF cote client : un acheteur malveillant peut techniquement bypass
- PDF generes cote client : pas de garantie d'integrite contenu vs trick navigateur
- Pas d'upload des pieces justificatives (CIP, ADC, plan topographique)
- Pas de notifications email aux parties au changement de statut

### 9.2 Roadmap V1 production (post-hackathon)

| Chantier V1                                       | Effort estime | Criticite |
|---------------------------------------------------|---------------|-----------|
| RLS strictes par role (chef, mairie, admin, anon) | 1 jour        | CRITIQUE  |
| Edge Function submit-dossier (validation serveur) | 1 jour        | CRITIQUE  |
| Upload des pieces justificatives en base          | 2 jours       | Haute     |
| Notifications email Supabase Auth                 | 0.5 jour      | Haute     |
| Provider SMS pour OTP citoyens (Twilio/MTN)       | 1 jour        | Haute     |
| Generation PDF cote serveur (Edge Function)       | 1 jour        | Moyenne   |
| Pagination et recherche full-text                 | 0.5 jour      | Moyenne   |
| Soft delete + archivage                           | 0.5 jour      | Basse     |
| Stats agregees anti-fraude (section 7.3 dossier)  | 2 jours       | Roadmap+  |
| Integration CEC et numero unique parcellaire      | 1 semaine     | Roadmap+  |

### 9.3 Conformite reglementaire visee

Le schema et les services anticipent l'alignement avec :

- Loi 2017-15 (Code Foncier et Domanial consolide)
- Decret 2025-176 du 9 avril 2025 (numero unique parcellaire, CEC)
- Loi 2025-05 du 11 mars 2025 (loi-cadre construction et habitation)
- Reforme PPMEC en cours (Projet de Preparation a la Mise a l'Echelle du Cadastre)

## 10. Pourquoi Gandehou au Benin

---

### 10.1 Le probleme reel

Au Benin, l'insecurite fonciere est l'un des sujets les plus douloureux du quotidien. Les conflits autour de la terre representent une part majeure des affaires civiles devant les tribunaux. Les causes sont connues : multiplicité des actes de presumption (Attestation de Detention Coutumiere, certificat administratif, permis d'habiter, anciennes conventions de vente sous seing prive), absence d'un cadastre national exhaustif, et surtout pratique persistante des doubles ventes ou un meme terrain est cede successivement a plusieurs acheteurs de bonne foi.

Pour une famille moyenne, l'achat d'une parcelle represente souvent l'investissement d'une vie. Une falsification ou une contestation peut signifier la perte totale de cet investissement, des annees de procedure judiciaire, et dans les cas extremes une demolition forcee de la construction. Le Conseil des Ministres du 4 mars 2026 a d'ailleurs declare 22 perimetres d'utilite publique sur le fondement de l'article 529 CFD, precisement parce que des decisions de justice mettaient en peril des constructions massives a Cotonou, Abomey-Calavi, Klouekanmey, Bopa, Zogbodomey et Lalo.

### 10.2 Ce que la reforme 2013-2017 a fait

La Loi n. 2013-01 du 14 aout 2013 a institue le Code Foncier et Domanial et cree l'Agence Nationale du Domaine et du Foncier (ANDF), qui devient le guichet unique. La Loi n. 2017-15 a prolonge la periode transitoire jusqu'au 14 aout 2023, date depuis laquelle le recours au Notaire et le Titre Foncier delivre par l'ANDF sont les seules voies legales de formalisation d'une vente.

Cette modernisation est juridiquement solide mais cree un goulot d'etranglement reel : la confirmation prealable des droits (etape obligatoire pour les terrains non encore titres, soit la majorite du foncier rural et peri-urbain) est longue, couteuse, et hors de portee economique pour beaucoup de menages. Le decret n. 2025-176 du 9 avril 2025 a tente de fluidifier en reintroduisant les communes via le Certificat d'Enregistrement au Cadastre (CEC), avec l'appui du consortium PPMEC (Ministere de l'Economie et des Finances, VNG International, Kadaster International).

### 10.3 Ou Gandehou s'insere

Gandehou ne pretend pas remplacer le Notaire ni l'ANDF. Notre couche se positionne en complement de confiance, applicable a CHAQUE etape deja existante du circuit legal : attestation de detention coutumiere, dossier de bornage par geometre agree, acte notarie, Titre Foncier, et demain le CEC. Chaque document officiel ou pre-officiel est hashe et horodate sur Bitcoin des sa creation. Toute modification ulterieure devient detectable et prouvable publiquement, gratuitement, par n'importe qui (banque, acheteur, notaire, agent ANDF).

Concretement : un acheteur qui detient un PDF de son acte peut le glisser-deposer sur la page publique de Gandehou. En quelques secondes, il sait si le document a ete altere depuis son ancrage initial. Un notaire peut faire la meme verification avant de contre-signer une transaction sur un terrain a historique complexe. Une banque peut verifier en lot des centaines d'actes avant d'octroyer des prets hypothecaires.

### 10.4 Pourquoi Bitcoin et pas une base SQL classique

- Incorruptibilite temporelle : aucun administrateur, meme malveillant, ne peut alterer un hash ancre il y a 30 ans

- Survie independante : si les serveurs Gandehou s'eteignent, n'importe quel outil OpenTimestamps tiers continue de decoder la preuve
- Cout reseau zero : l'agregation Merkle d'OpenTimestamps permet d'ancrer des milliers de hashes dans une seule transaction Bitcoin (frais reseau partages, gratuit en pratique)
- Argument souverainete : le Benin n'a pas besoin de faire confiance a un fournisseur cloud etranger ; la preuve vit sur un reseau monetaire decentralise mondial

## 10.5 Cible utilisateurs

Gandehou est concu pour une population mixte : citoyens souvent illettres ou faiblement scolarises (40% de la population beninoise selon les statistiques nationales), agents fonciers communaux, notaires, banques, et a terme l'ANDF elle-meme via integration directe. L'interface s'appuie sur des codes visuels universels (vert/jaune/rouge inspires du drapeau national) et le QR code permet une verification meme sans saisie clavier.

Le scenario de reference que nous portons : Maman Chantal, vendeuse de poisson a Dantokpa, achete une parcelle a Abomey-Calavi. L'agent local saisit avec elle dans Gandehou, le PDF est ancre. Deux ans plus tard, le vendeur initial tente une seconde vente avec un document altere du meme dossier. Le second acheteur scanne le QR : alerte rouge instantanee. La preuve originale est recevable devant le tribunal sans dependre de la disponibilite d'un serveur.

## 10.6 Modele economique et viabilite

Gandehou est concu pour etre gratuit pour le citoyen final (verification publique sans compte). Le modele de revenus repose sur trois piliers : (1) API premium pour les banques et notaires qui font de la verification en masse, (2) partenariat institutionnel avec l'ANDF et les communes pour integration au circuit officiel (financement public via PPMEC ou bailleurs), (3) services additionnels (notifications WhatsApp, signatures biometriques avancees, archivage longue duree).

## 10.7 Ce qui rend ce projet specifiquement beninois

- Cadrage juridique strict : le projet cite explicitement les articles du CFD et le decret 2025-176
- Codes visuels du drapeau national (vert/jaune/rouge), conviction esthetique africaine
- Adaptation a la realite informelle : le Chef de Quartier (qui n'a aucun statut legal dans le CFD) est positionne comme couche sociale de pre-enregistrement, sans pretendre se substituer au circuit officiel
- Anticipation de la reforme PPMEC en cours : compatibilite documentee avec le futur numero unique parcellaire
- Scenarios construits sur des localites reelles : Cotonou, Abomey-Calavi, Cocotomey, Dantokpa

# Conclusion

---

Gandehou est aujourd'hui un POC backend fonctionnel, deploye en production sur Supabase, avec un ancrage Bitcoin reellement teste de bout en bout, une detection de falsification verifiee, et un alignement juridique serieux sur le Code Foncier et Domanial beninois ainsi que sur la reforme 2025-2026 en cours (decret 2025-176).

L'ecart entre un POC hackathon (etat actuel) et une V1 production (cible 3-6 mois) est documente explicitement dans la section 9 ci-dessus. Les briques critiques (ancrage Bitcoin reel, schema SQL, services typees, vrification cryptographique, audit trail automatique) sont en place et utilisables des aujourd'hui. Les briques restantes (RLS strictes, validation serveur exhaustive, pieces justificatives, notifications) sont identifiees, priorisees, et accessibles en effort raisonnable.

Notre conviction : Gandehou n'est pas un gadget blockchain plaque sur un probleme generique. C'est une reponse honnete et techniquement precise a un probleme reel, documente par l'ANDF elle-meme comme un defi majeur de la reforme en cours. Bitcoin et OpenTimestamps sont ici utilises non pour leur cote spectaculaire, mais parce qu'ils sont l'outil le plus pertinent pour la partie 'preuve d'integrite' du probleme : preuve gratuite, publique, decentralisee, et capable de survivre a la disparition de son operateur.

## POUR LE JURY BITCOIN MASTERMIND 2026

Nous demontrons une chaine end-to-end reelle : un PDF cree, hashe en SHA-256, soumis au calendrier OpenTimestamps, agrege dans un bloc Bitcoin, et verifiable cryptographiquement par n'importe qui sans dependre de notre infrastructure. La falsification est detectee en temps reel. Le cadre juridique beninois est respecte. La roadmap vers la production est concrete et chiffee.

Cotonou, le 30 juin 2026. Equipe Gandehou.